

Projet C Avancé (L3 GL / UDBL 25-26) — Logiciel de Traitement de Texte Intelligent en C

Plateforme cible : Windows 10 / 11 (x64)

Langage : C (C11 standard)

Mode de fonctionnement : 100 % hors ligne (offline)

Table des matières

- [Projet C Avancé \(L3 GL / UDBL 25-26\) — Logiciel de Traitement de Texte Intelligent en C](#)
 - [Table des matières](#)
 - [1. Présentation du projet](#)
 - [2. Objectifs et périmètre](#)
 - [2.1 Objectifs principaux](#)
 - [2.2 Périmètre inclus](#)
 - [2.3 Périmètre exclu](#)
 - [3. Architecture générale](#)
 - [3.1 Principes architecturaux](#)
 - [4. Modules fonctionnels](#)
 - [4.1 Éditeur de texte](#)
 - [4.2 Interface utilisateur Win32 + Scintilla](#)
 - [4.3 Moteur NLP — Correction française](#)
 - [4.4 Moteur LLM embarqué](#)
 - [4.5 Moteur de règles métier](#)
 - [4.6 Couche système et fichiers](#)
 - [5. Stack technique et dépendances](#)
 - [6. Format du fichier de règles](#)
 - [7. Modèles LLM recommandés](#)
 - [8. Répartition des tâches — 4 développeurs](#)
 - [Responsabilité de l'équipe](#)
 - [Phases et objectifs](#)
 - [Responsabilités par module](#)
 - [DEV-A — Infrastructure & Éditeur de texte](#)
 - [DEV-B — Interface utilisateur Win32 + Scintilla](#)
 - [DEV-C — LLM & Moteur NLP](#)
 - [DEV-D — Moteur de règles & Intégration générale](#)
 - [9. Collaboration, rendu et évaluation](#)
 - [9.1 Gestion du code source avec Git / GitHub](#)
 - [9.2 Rapport à remettre](#)
 - [9.3 Critères d'évaluation](#)
-

1. Présentation du projet

Le projet consiste à développer un logiciel de traitement de texte intelligent, baptisé **IntelliEditor**, développé en langage C, destiné à fonctionner entièrement hors ligne sur Windows. Il combine les fonctionnalités classiques d'un éditeur de texte avec des capacités d'intelligence artificielle intégrées directement dans l'application, sans recours à un serveur distant ou à une connexion Internet.

Le logiciel est conçu pour accompagner les étudiants finalistes et chercheurs dans la rédaction de documents structurés (mémoires, rapports, articles), en leur offrant une assistance active à chaque étape : correction linguistique, reformulation stylistique, et respect automatisé de règles de rédaction définies à l'avance.

2. Objectifs et périmètre

2.1 Objectifs principaux

Priorité	Objectif
P1	Fournir un éditeur de texte fonctionnel avec mise en forme de base
P1	Intégrer une correction orthographique et grammaticale en français
P1	Permettre le chargement d'un fichier de règles de rédaction personnalisables
P1	Effectuer le suivi en temps réel de l'application des règles chargées
P2	Proposer des reformulations stylistiques via un LLM local
P2	Fonctionner entièrement hors ligne, sans aucune dépendance réseau
P3	Proposer des résumés de sections et des suggestions de structure

2.2 Périmètre inclus

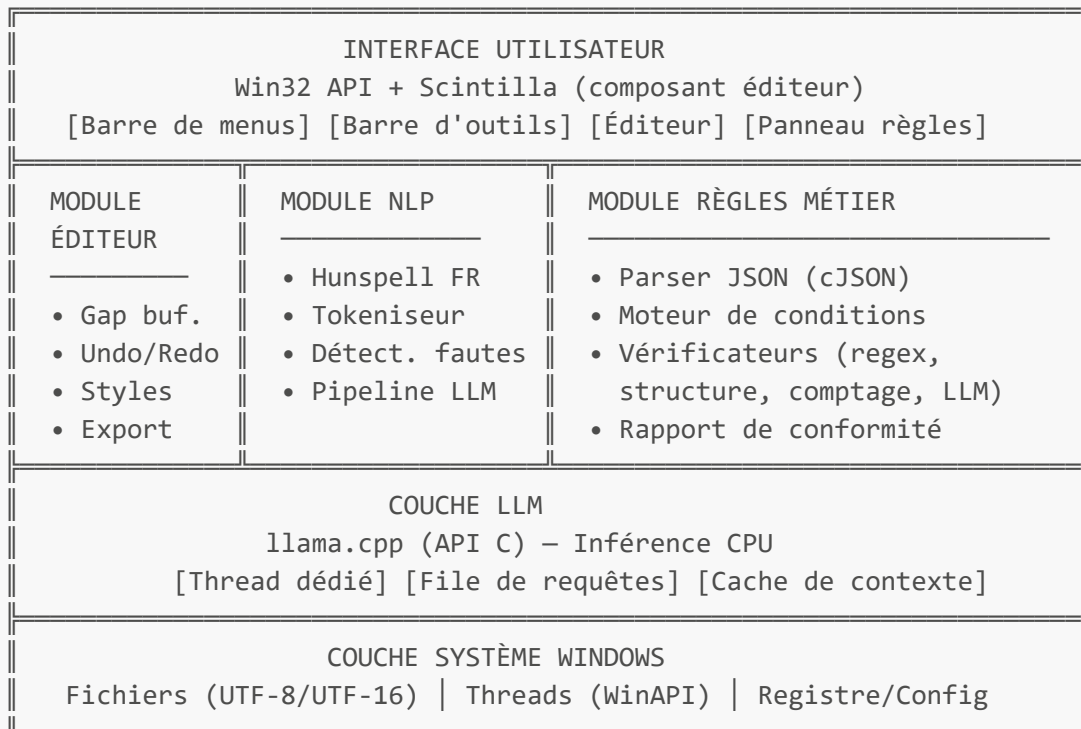
- Édition de texte riche (gras, italique, titres H1–H4, listes, alignement)
- Correction orthographique en temps réel (soulignement rouge)
- Détection de fautes grammaticales courantes en français
- Reformulation de phrases à la demande (via LLM)
- Chargement d'un fichier JSON (ou autre) de règles de rédaction
- Panneau de suivi des règles avec statut (respectée / non respectée / avertissement)
- Export en `.txt` et `.rtf`, import de `.txt`
- Gestion de documents multiples par onglets
- Interface entièrement en français

2.3 Périmètre exclu

- Collaboration en temps réel (multi-utilisateurs)
- Synchronisation cloud ou sauvegarde distante
- Support de `.docx` natif en écriture (lecture possible via conversion)
- Interface mobile ou web
- Support multilingue autre que le français

- Correcteur de plagiat

3. Architecture générale



3.1 Principes architecturaux

- **Séparation des responsabilités** : chaque module expose une API C claire ; les modules ne s'appellent pas directement en dehors de cette API.
- **Asynchronisme** : toutes les requêtes LLM (lentes) s'exécutent dans un thread séparé. L'UI n'est jamais bloquée.
- **UTF-8 interne** : tout le pipeline interne utilise UTF-8. La conversion vers UTF-16 (WinAPI) se fait uniquement aux points d'entrée/sortie système.
- **Tolérance aux pannes** : si le LLM est indisponible ou trop lent, les fonctionnalités de base (édition, Hunspell) restent pleinement opérationnelles.

4. Modules fonctionnels

4.1 Éditeur de texte

Le cœur de l'édition repose sur un **gap buffer**, structure de données standard des éditeurs de texte professionnels (utilisée par Emacs, entre autres). Elle permet des insertions et suppressions en $O(1)$ à la position courante du curseur.

Fonctionnalités couvertes :

- Insertion, suppression, sélection de texte
- Historique undo/redo illimité (pattern Command)

- Gestion des styles inline : **BOLD**, *ITALIC*, UNDERLINE, **HEADING_1..4**, **LIST_BULLET**, **LIST_NUMBERED**
- Numérotation automatique des titres et des listes
- Rechercher / Remplacer (avec regex optionnel)
- Statistiques en temps réel : nombre de mots, de caractères, de paragraphes

Export supporté :

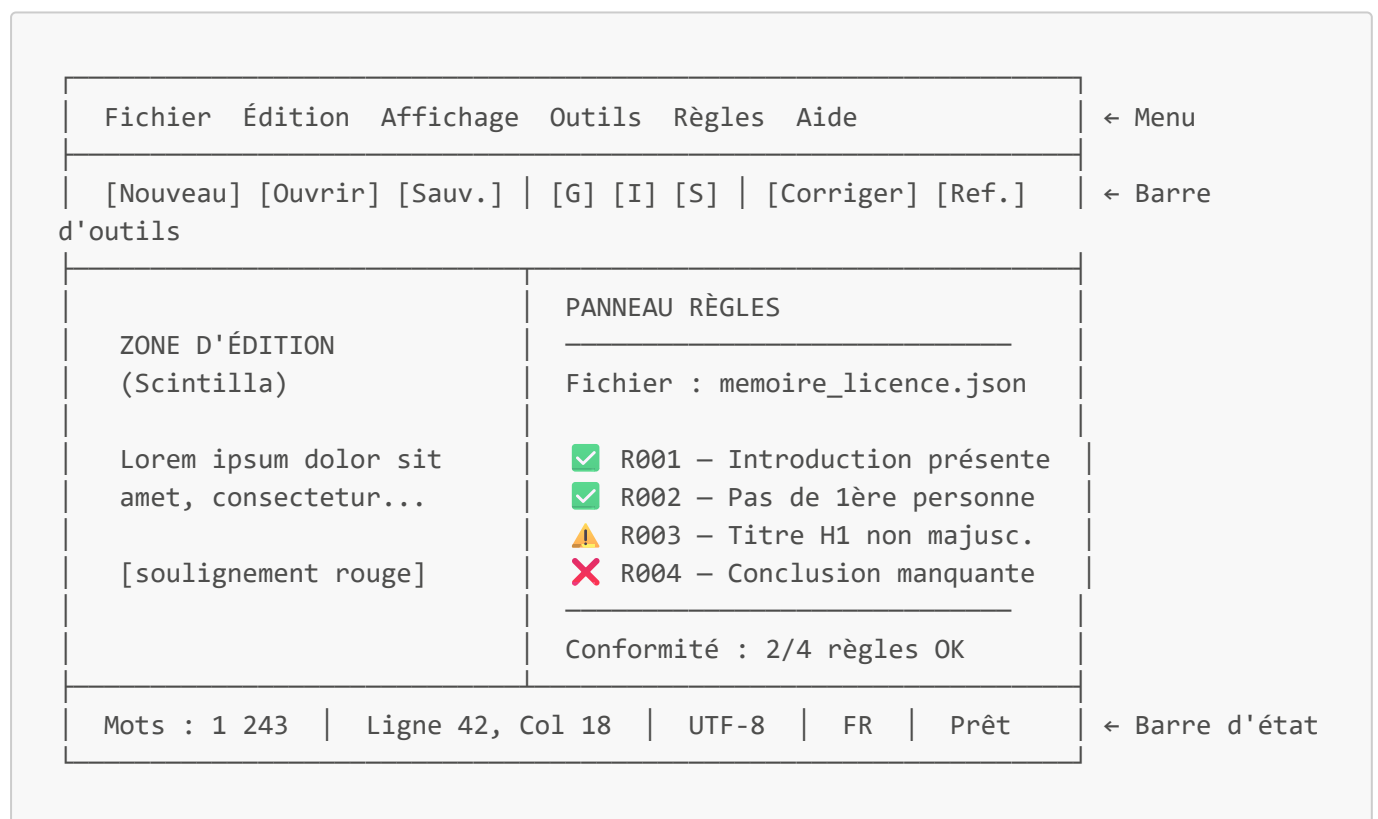
Format	Méthode
.txt	Écriture directe UTF-8
.rtf	Génération manuelle des balises RTF
.ie	Format binaire propriétaire (avec styles)

4.2 Interface utilisateur Win32 + Scintilla

L'interface est construite avec **l'API Win32 native** de Windows, sans framework supplémentaire. Le composant d'édition est **Scintilla** (la bibliothèque open source qui propulse Notepad++ et de nombreux IDE). Elle gère nativement :

- La coloration syntaxique et les indicateurs (soulignements, marges)
- L'affichage des annotations et marques d'erreur inline
- Le glisser-déposer de fichiers
- La numérotation des lignes et le pliage de sections

Disposition de la fenêtre principale :



4.3 Moteur NLP — Correction française

Le moteur NLP fonctionne en deux niveaux complémentaires :

Niveau 1 — Hunspell (rapide, synchrone)

Hunspell est une bibliothèque C open source avec dictionnaires français officiels. Elle gère :

- Correction orthographique mot par mot
- Suggestions de remplacement
- Gestion des mots composés et des apostrophes françaises

Niveau 2 — LLM (approfondi, asynchrone)

Pour la grammaire et la reformulation, une requête est envoyée au LLM avec un prompt structuré. La réponse est attendue de manière non bloquante.

Types d'erreurs détectées :

Catégorie	Source	Exemple
Orthographe	Hunspell	"langague" → "langage"
Accord sujet-verbe	LLM	"les règles est" → "les règles sont"
Accord en genre/nombre	LLM	"une bonne résultat"
Ponctuation française	Règles codées	Espace avant :, ?, !, ;
Répétition de mots	Règles codées	Même mot répété à moins de 3 mots d'écart
Anglicismes courants	Liste embarquée	"un mail" → "un courriel"

4.4 Moteur LLM embarqué

Le LLM est intégré via **llama.cpp**, dont l'API C est stable et maintenue activement. L'inférence tourne entièrement sur CPU.

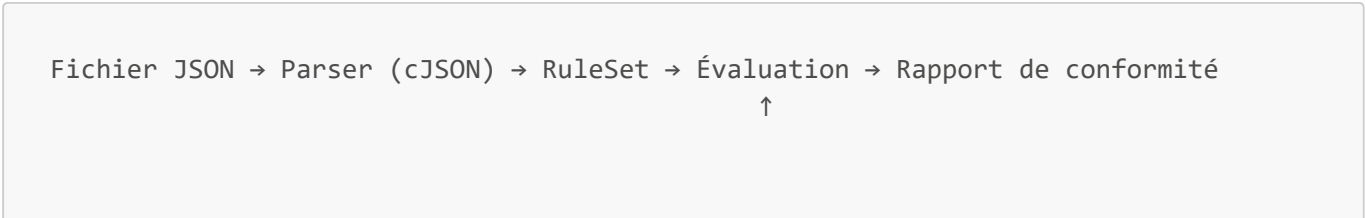
Gestion des performances :

- Le modèle est chargé une seule fois au démarrage (5–15 secondes selon le modèle)
- Un indicateur de chargement est affiché dans la barre de statut
- Les requêtes sont mise en file et traitées séquentiellement
- Un timeout de 30 secondes par requête est appliqué
- Le contexte KV est réutilisé entre requêtes similaires

4.5 Moteur de règles métier

C'est la fonctionnalité différenciante du logiciel. L'utilisateur charge un fichier JSON décrivant les règles de rédaction applicables à son type de document.

Cycle de vie d'une règle :



Déclenchée à chaque modification significative
(après 2 secondes d'inactivité)

Vérificateurs implémentés :

Type de vérificateur	Description	Exemple d'usage
section_exists	Détecte la présence d'un titre de section	"Introduction" obligatoire
section_order	Vérifie l'ordre des sections	Abstract avant Introduction
word_count_min	Nombre de mots minimum dans une section	Introduction ≥ 300 mots
word_count_max	Nombre de mots maximum	Résumé ≤ 250 mots
regex_forbidden	Expression régulière interdite	Première personne du singulier
regex_required	Expression régulière obligatoire	Mention de l'établissement
heading_format	Format attendu des titres	H1 en majuscules
citation_present	Présence d'une section bibliographie	Référence obligatoire
llm_semantic	Vérification sémantique via LLM	"La problématique est-elle posée ?"

Panneau de conformité :

Chaque règle affiche l'un de ces statuts :

- ✔ **Conforme** — La règle est respectée
- ⚠ **Avertissement** — La règle est partiellement respectée ou non vérifiable
- ✖ **Non conforme** — La règle est clairement violée
- 🔄 **En cours** — Vérification LLM en attente

Un clic sur une règle non conforme **positionne le curseur** à l'endroit concerné dans le document.

4.6 Couche système et fichiers

Gestion de la configuration :

La configuration est stockée dans %APPDATA%\IntelliEditor\config.ini au format INI (parsé manuellement en C).

```
[General]
language=fr_FR
theme=light
autosave_interval=300

[LLM]
model_path=C:\Program Files\IntelliEditor\models\mistral-7b-q4.gguf
```

```
n_threads=4
n_ctx=4096

[Editor]
font_name=Consolas
font_size=12
line_numbers=1
word_wrap=1
```

5. Stack technique et dépendances

Composant	Bibliothèque	Rôle
Compilateur	GCC (MinGW-w64)	Compilation C11 sur Windows
Build system	Make / CMake	Gestion de la compilation
Interface graphique	Win32 API	Fenêtres, contrôles natifs
Éditeur texte	Scintilla	Composant éditeur avancé
LLM inférence	llama.cpp	Inférence de modèles GGUF
Correcteur ortho	Hunspell	Dictionnaire français
Parsing JSON	cJSON	Fichiers de règles
Expressions régulières	PCRE2	Vérificateurs regex
Compression	zlib	Import/export de fichiers
Tests unitaires	cmocka	Tests des modules C

Environnement de développement recommandé :

- MSYS2 + MinGW-w64 (toolchain principale)
 - VS Code ou CLion comme IDE
 - Git pour le contrôle de version
 - Valgrind (via WSL) pour la détection de fuites mémoire
-

6. Format du fichier de règles

Les fichiers de règles sont au format **JSON**. Voici un exemple pour un mémoire de licence :

```
{
  "meta": {
    "document_type": "Mémoire de Licence",
    "version": "1.0",
    "author": "Université Don Bosco de Lubumbashi",
    "description": "Règles de rédaction pour les mémoires de Licence –  
promotion 2025-2026"
  },
  "rules": [
    {
      "id": "R001",
      "category": "structure",
      "severity": "error",
      "description": "Le document doit contenir une section Introduction",
      "check_type": "section_exists",
      "parameter": "Introduction"
    },
    {
      "id": "R002",
      "category": "structure",
      "severity": "error",
      "description": "Le document doit contenir une section Conclusion",
      "check_type": "section_exists",
      "parameter": "Conclusion"
    },
    {
      "id": "R003",
      "category": "structure",
      "severity": "warning",
      "description": "Les sections doivent apparaître dans l'ordre attendu",
      "check_type": "section_order",
      "parameter": [
        "Résumé",
        "Introduction",
        "Revue de littérature",
        "Méthodologie",
        "Résultats",
        "Discussion",
        "Conclusion",
        "Bibliographie"
      ]
    },
    {
      "id": "R004",
      "category": "style",
      "severity": "warning",
      "description": "Éviter l'emploi de la première personne du singulier",
      "check_type": "regex_forbidden",
```



```

        "parameter": "\\bje\\b|\\bj'|\\bme\\b|\\bmon\\b|\\bma\\b|\\bmes\\b",
        "flags": "case_insensitive"
    },
    {
        "id": "R005",
        "category": "format",
        "severity": "warning",
        "description": "Les titres de niveau 1 doivent être entièrement en
majuscules",
        "check_type": "heading_format",
        "parameter": { "level": 1, "case": "uppercase" }
    },
    {
        "id": "R006",
        "category": "longueur",
        "severity": "warning",
        "description": "L'introduction doit contenir au moins 300 mots",
        "check_type": "word_count_min",
        "parameter": { "section": "Introduction", "min_words": 300 }
    },
    {
        "id": "R007",
        "category": "longueur",
        "severity": "warning",
        "description": "Le résumé ne doit pas dépasser 250 mots",
        "check_type": "word_count_max",
        "parameter": { "section": "Résumé", "max_words": 250 }
    },
    {
        "id": "R008",
        "category": "structure",
        "severity": "error",
        "description": "Une section Bibliographie doit être présente",
        "check_type": "section_exists",
        "parameter": "Bibliographie"
    },
    {
        "id": "R009",
        "category": "sémantique",
        "severity": "info",
        "description": "La problématique de recherche doit être explicitement
posée",
        "check_type": "llm_semantic",
        "parameter": "Le texte de la section Introduction pose-t-il
explicitement une problématique ou une question de recherche ?",
        "target_section": "Introduction"
    }
]
}

```

7. Modèles LLM recommandés

Modèle (format GGUF)	Taille	RAM requise	Qualité FR
Mistral 7B Instruct v0.3	4.1 Go	~6 Go	5/5
Qwen2.5 7B Instruct	4.4 Go	~6 Go	5/5
Phi-3 Mini 3.8B	2.2 Go	~4 Go	4/5
Qwen2.5 3B Instruct	1.9 Go	~3 Go	4/5
Llama 3.2 3B Instruct	2.0 Go	~3.5 Go	3/5

Le modèle peut être sélectionné dans les préférences du logiciel. Un assistant de configuration au premier lancement guide l'utilisateur selon la RAM disponible.

8. Répartition des tâches — 4 développeurs

Responsabilité de l'équipe

Dev	Identifiant	Rôle
Développeur 1	DEV-A	Spécialiste systèmes / bas niveau C
Développeur 2	DEV-B	Spécialiste UI / Win32 / expérience utilisateur
Développeur 3	DEV-C	Spécialiste IA / NLP / intégration LLM
Développeur 4	DEV-D	Développeur fullstack C / intégration / tests

Phases et objectifs

Phase	Titre	Objectif de livraison
Phase 1	Fondations	Éditeur fonctionnel + LLM chargé
Phase 2	Intelligence de base	Correction ortho + règles structurelles
Phase 3	Fonctionnalités avancées	LLM grammaire + règles sémantiques
Phase 4	Stabilisation	Version 1.0 distribuable

Responsabilités par module

DEV-A — Infrastructure & Éditeur de texte

Responsable des fondations techniques sur lesquelles tout le reste s'appuie.

Tâche	Module	Phase
Mise en place CMake, MSYS2, toolchain MinGW	Infrastructure	1

Tâche	Module	Phase
Implémentation du gap buffer	editor/gap_buffer	1
Système undo/redo (pattern Command)	editor/undo_redo	1
Couche UTF-8 / UTF-16 (encoding.c)	utils/encoding	1
Gestion configuration INI	utils/config	1
Moteur de formatage des styles	editor/formatter	2
Export TXT, RTF, format propriétaire	editor/exporter	2
Rechercher / Remplacer	editor/search_replace	2
Optimisation mémoire, allocateur debug	utils/memory	3
Packaging Inno Setup	installer/	4

DEV-B — Interface utilisateur Win32 + Scintilla

Responsable de tout ce que l'utilisateur voit et touche.

Tâche	Module	Phase
Fenêtre principale Win32 (WinMain, WndProc)	ui/main_window	1
Intégration et configuration de Scintilla	ui/scintilla_wrapper	1
Barre de menus et raccourcis clavier	ui/main_window	1
Barre d'outils (toolbar)	ui/toolbar	2
Barre de statut (mots, ligne, encodage)	ui/statusbar	2
Panneau latéral des règles	ui/rules_panel	2
Boîtes de dialogue (ouvrir, sauvegarder, config)	ui/dialogs	2
Affichage inline des erreurs NLP dans Scintilla	ui/scintilla_wrapper	3
Thème clair/sombre	ui/	3
Glisser-déposer de fichiers et de règles	ui/main_window	3

DEV-C — LLM & Moteur NLP

Responsable de toute l'intelligence artificielle et du traitement linguistique.

Tâche	Module	Phase
Intégration llama.cpp (compilation, liaison)	llm/llm_interface	1
Test de charge d'un modèle GGUF	llm/llm_interface	1
Thread worker LLM asynchrone	llm/llm_thread	1

Tâche	Module	Phase
File de requêtes LLM (queue thread-safe)	llm/llm_thread	1
Intégration Hunspell + dictionnaires fr_FR	nlp/hunspell_wrap	2
Tokeniseur de phrases et de mots	nlp/tokenizer	2
Pipeline de correction orthographique	nlp/nlp_engine	2
Templates de prompts (grammaire, reformulation)	llm/prompts	2
Parser JSON des réponses LLM	llm/json_parser	2
Pipeline de vérification grammaticale LLM	nlp/nlp_engine	3
Fonctionnalité de reformulation	nlp/nlp_engine	3
Optimisation : cache KV, batch LLM	llm/llm_interface	4

DEV-D — Moteur de règles & Intégration générale

Responsable du moteur de règles métier et de la cohésion de tous les modules.

Tâche	Module	Phase
Parser de fichiers JSON de règles (cJSON)	rules/rule_parser	1
Structures de données RuleSet, Rule, RuleResult	rules/rule_engine	1
Vérificateur <code>section_exists</code>	rules/checkers/section_checker	2
Vérificateur <code>regex_forbidden/required</code> (PCRE2)	rules/checkers/regex_checker	2
Vérificateur <code>word_count_min/max</code>	rules/checkers/count_checker	2
Vérificateur <code>section_order</code>	rules/checkers/section_checker	2
Rapport de conformité (RuleReport)	rules/rule_report	2
Vérificateur <code>llm_semantic</code>	rules/checkers/llm_checker	3
Vérificateur <code>heading_format</code>	rules/checkers/section_checker	3
Intégration NLP → UI (signaux d'erreur vers Scintilla)	Intégration	3
Intégration Règles → Panneau UI	Intégration	3
Rédaction des fichiers de règles templates	data/rule_templates/	3
Tests unitaires (cmocka) tous modules	tests/	4

9. Collaboration, rendu et évaluation

9.1 Gestion du code source avec Git / GitHub

Le projet **doit obligatoirement** être versionné et collaboré via **Git** et hébergé sur un dépôt **public GitHub**.

- Un seul dépôt par groupe de 4 développeurs.
- Chaque développeur travaille sur sa **propre branche** (nommée par exemple **dev-a**, **dev-b**, **dev-c**, **dev-d**) et intègre ses modifications via des **Pull Requests** vers la branche **main**.
- Les commits doivent être **réguliers, atomiques et bien décrits** — un historique vide ou avec un seul commit massif en fin de projet sera pénalisé.
- Le fichier **README.md** à la racine du dépôt doit contenir au minimum : le nom du projet, les noms des membres de l'équipe et les instructions de compilation.

9.2 Rapport à remettre

Un rapport PDF sera déposé à la fin du projet. Il devra contenir :

Section	Contenu attendu
Lien du dépôt GitHub	URL complète du dépôt public
Fonctionnalités implémentées	Liste des fonctionnalités réellement développées et fonctionnelles
Fonctionnalités non implémentées	Liste des fonctionnalités prévues mais non réalisées, avec justification
Difficultés rencontrées	Problèmes techniques, choix abandonnés, solutions alternatives adoptées
Répartition effective des tâches	Ce que chaque développeur a concrètement réalisé (peut différer du plan initial)
Informations complémentaires	Tout élément jugé utile : ressources utilisées, outils, remarques sur le projet

9.3 Critères d'évaluation

L'évaluation est **individuelle**, même si le projet est collectif.

Critère	Poids
Structure du code et qualité du logiciel	~30 %
Exposé individuel oral	~70 %

L'exposé oral est le critère **principal**. Chaque étudiant devra expliquer clairement, devant l'enseignant, le code qu'il a écrit : les choix techniques effectués, le fonctionnement des algorithmes et structures de données utilisés, et les difficultés surmontées. Un étudiant incapable d'expliquer son propre code obtiendra une note très faible, indépendamment de la qualité du logiciel livré.

L'exposé sera organisé à la fin du cours. La date exacte sera communiquée ultérieurement.